

CC-0F4

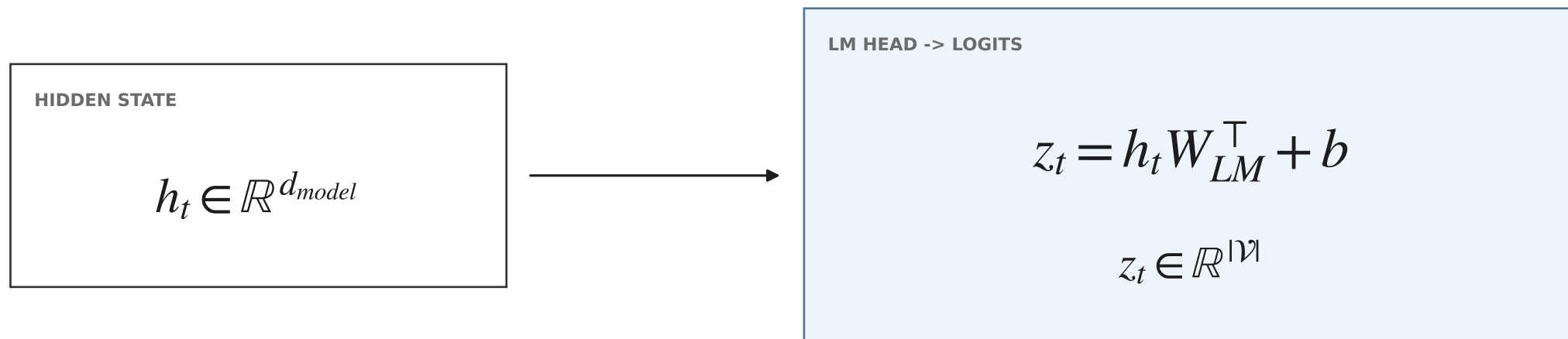
Ecuaciones esenciales

Semana 2

Inferencia, logits, decoding, contexto y KV cache

1. De la representación contextual a los logits

El Transformer produce un hidden state; el LM head lo proyecta al vocabulario.



Los logits pueden ser positivos o negativos y no tienen que sumar 1.

Lectura: logit = score no normalizado; todavía no es una probabilidad.

2. Logits -> softmax -> distribución del siguiente token

Softmax convierte scores en una distribución sobre el vocabulario.

PROBABILIDAD

$$p(v | x_{1:t}) = \frac{e^{z_v}}{\sum_{u \in \mathcal{V}} e^{z_u}}$$

INVARIANTE

$$\sum_{v \in \mathcal{V}} p(v | x_{1:t}) \approx 1$$

Softmax normaliza. No decide causalidad y tampoco decide qué token será elegido.

Lectura: modelo causal -> logits -> distribución; decoding actúa después.

3. Temperatura y entropía

La temperatura cambia la concentración de la distribución antes de muestrear.

TEMPERATURA

$$p_{\tau}(v) = \frac{e^{z_v/\tau}}{\sum_u e^{z_u/\tau}}$$

ENTROPÍA

$$H(p) = -\sum_v p(v) \log_2 p(v)$$

$\tau < 1$ -> más concentrada $\tau = 1$ -> original $\tau > 1$ -> más plana

Lectura: en el Experimento A, temperatura cambia la política de decoding, no los pesos del modelo.

4. Greedy decoding frente a sampling

Una misma distribución puede convertirse en secuencias distintas según la regla de selección.

GREEDY

$$\hat{x}_{t+1} = \arg \max_{v \in \mathcal{V}} p_t(v)$$

Determinista para logits fijos.

SAMPLING

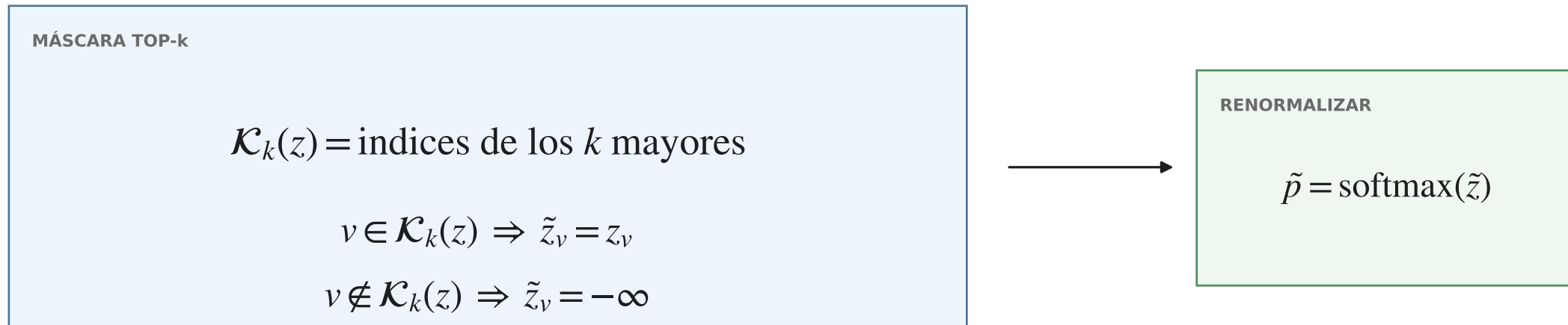
$$x_{t+1} \sim \text{Categorical}(p_t)$$

Estocástico: la seed y la distribución importan.

Lectura: greedy elige el máximo; sampling extrae una muestra de la distribución.

5. Top-k: conjunto fijo de candidatos

Se conservan los k logits mayores y se enmascara el resto.



Lectura: top-k fija cuántos candidatos sobreviven; luego softmax vuelve a normalizar.

6. Top-p: núcleo de masa acumulada

El número de candidatos es dinámico: depende de la forma de la distribución.

ORDENAR Y ACUMULAR

$$p_{(1)} \geq p_{(2)} \geq \dots \geq p_{(|V|)}$$
$$m^* = \min \left\{ m : \sum_{j=1}^m p_{(j)} \geq p_0 \right\}$$

NÚCLEO

$$\mathcal{N}_{p_0} = \{(1), \dots, (m^*)\}$$

Conservar -> renormalizar -> muestrear.

Lectura: top-p conserva la cantidad mínima de tokens cuya masa acumulada alcanza el umbral.

7. Autoregresión: generar un token y ampliar el contexto

La salida vuelve a entrar como parte del prefijo del siguiente paso.

FACTORIZACIÓN AUTOREGRESIVA

$$P(x_1:T) = \prod_{t=1}^{T-1} P(x_{t+1} | x_1:t)$$

Paso t: $x_1 \dots x_t \rightarrow \text{logits } z_t \rightarrow p_t \rightarrow \text{decoding} \rightarrow x_{\{t+1\}} \rightarrow \text{nuevo prefijo}$

Lectura: la generación completa es una secuencia de decisiones locales del siguiente token.

8. Ventana de contexto como presupuesto

La longitud relevante es la longitud tokenizada, no el número de palabras.

RESTRICCIÓN

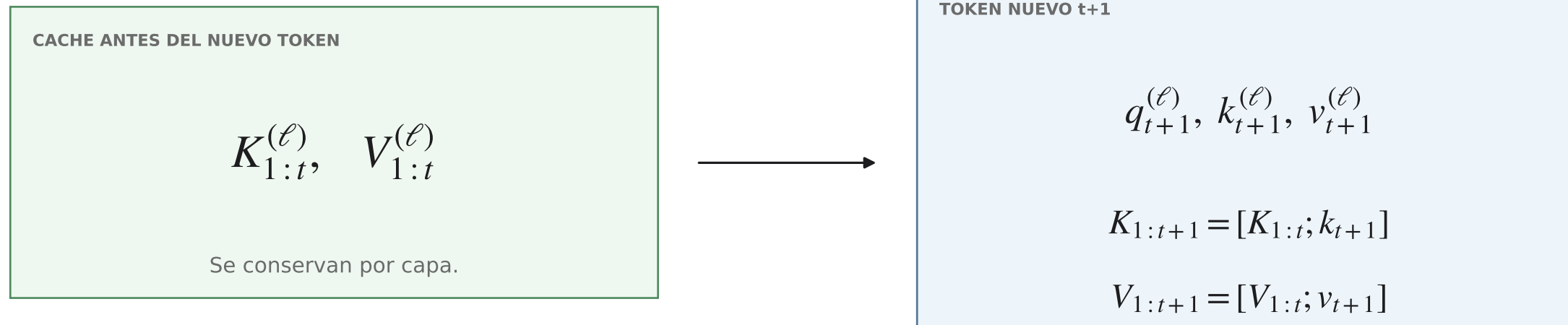
$$T_{prompt} + T_{historial} + T_{salida} \leq L_{max}$$

Más tokens -> más cómputo -> más estado K/V -> más memoria.

Lectura: la ventana de contexto obliga a decidir cuánto presupuesto asignar al prompt, historial y salida.

9. KV cache: qué se reutiliza

Para los tokens previos ya existen keys y values en cada capa causal.



Lectura: el cache intercambia memoria por menor recomputación de K/V del prefijo.

10. Attention durante decode con KV cache

El token nuevo consulta todas las keys permitidas, pero no reconstruye K/V anteriores.

UN SOLO QUERY NUEVO

$$s_{t+1} = \frac{q_{t+1} K_{1:t+1}^\top}{\sqrt{d_h}}$$

$$o_{t+1} = \text{softmax}(s_{t+1}) V_{1:t+1}$$

La causalidad sigue vigente: el nuevo token solo puede usar posiciones disponibles hasta t+1.

Lectura: KV cache no cambia la ecuación de attention; cambia qué tensores se reutilizan.

11. Memoria lógica del KV cache

Estimación didáctica para MHA, GQA y MQA bajo supuestos explícitos.

FÓRMULA

$$M_{KV} = B L T 2 H_{KV} d_h b$$

TÉRMINOS

B: batch

L: capas

T: tokens conservados

H_{KV} : KV heads

d_h : dimensión/head

b: bytes/elemento; 2 = K y V

$$GB = M/10^9, \quad GiB = M/2^{30}$$

Lectura: esta fórmula estima solo el estado K/V, no la memoria total del proceso de inferencia.

12. MHA, GQA y MQA: reducir KV heads

Si B , L , T , d_h y precisión se mantienen fijos, la memoria K/V escala con H_{KV} .

MHA

$$H_{KV} = H_Q$$

GQA

$$1 < H_{KV} < H_Q$$

MQA

$$H_{KV} = 1$$

RAZÓN DE MEMORIA K/V

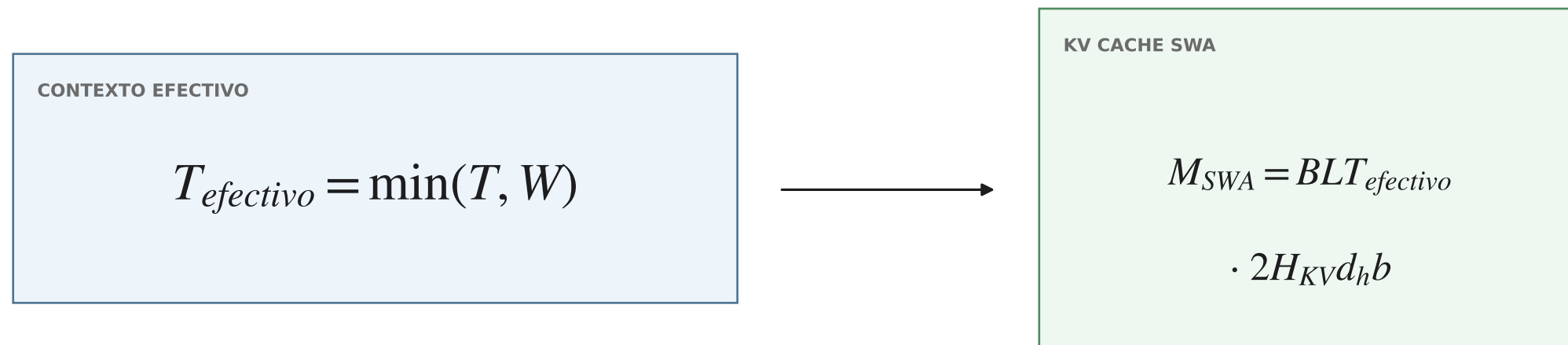
$$\frac{M_{GQA}}{M_{MHA}} = \frac{H_{KV}}{H_Q}$$

Con $H_Q = 32$ y $H_{KV} = 8$: $8/32 = 0.25 \rightarrow 25\%$ del componente K/V de MHA.

Lectura: GQA reduce el número de heads de K/V sin reducir el número de query heads.

13. Sliding Window Attention: limitar el contexto efectivo

Bajo cache truncado a la ventana activa, solo se conservan W posiciones por capa local.



Si $T \gg W$, la memoria lógica del cache deja de crecer con T bajo este supuesto.

Lectura: SWA reduce posiciones históricas visibles/conservadas; no reduce necesariamente el número de heads.

14. MLA conceptual: comprimir el estado K/V

Attention AI Lab usa un modelo didáctico de rango latente para estimar memoria.

MODELO CONCEPTUAL

$$M_{MLA} = B L T 2 r b$$

r = rango latente usado para representar el estado asociado a K/V.

NO MEZCLAR MODELOS DIDÁCTICOS DISTINTOS

La cifra depende de qué exactamente se almacena y de cómo se define la compresión.

Lectura: MLA y GQA reducen memoria mediante mecanismos diferentes: compresión latente vs menos KV heads.

15. Experimento B: memoria estimada del KV cache

Configuración fija: B=1, L=32, T=131072, H_Q=32, d_h=128, fp16/bf16 (2 bytes).

Variante	Supuesto clave	GB	GiB
MHA	H_KV=32	68.72	64.00
GQA	H_KV=8	17.18	16.00
SWA	W=4096, H_KV=32	2.15	2.00
MLA conceptual	r=512	8.59	8.00

Comparación válida solo bajo el mismo conjunto de supuestos y la fórmula declarada.

Lectura: memoria lógica estimada != latencia real != throughput != memoria pico física de GPU.

16. FlashAttention: misma atención, ejecución distinta

FlashAttention es exacta: reorganiza el cálculo para reducir tráfico de memoria y materialización intermedia.

ECUACIÓN MATEMÁTICA

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

La salida matemática objetivo no cambia.

DISTINCIÓN

FlashAttention

reduce I/O/intermedios

pero no reduce por sí sola
H_KV ni T del KV cache.

Lectura: no confundir optimización del kernel de attention con reducción lógica del KV cache.

17. Experimento A: decoding bajo una sola modificación

Modelo, prompt, max_new_tokens y protocolo se mantienen fijos; cambia una variable de decoding.

DISTRIBUCIÓN

$$H(p_t) = -\sum_v p_t(v) \log_2 p_t(v)$$

Concentración/incertidumbre del siguiente token.

DIVERSIDAD DE TEXTO

$$\text{distinct-}n = \frac{|U_n|}{|G_n|}$$

Puede usarse con n=1 y n=2.

A1: temperature=0.7 A2: 1.3 A3: top-k=3 A4: top-p=0.85

Lectura: una modificación por vez permite atribuir el cambio observado al factor estudiado.

18. Estimación analítica frente a benchmark físico

Dos preguntas diferentes requieren evidencia diferente.

ESTIMACIÓN ANALÍTICA

$$M_{KV} = BLT 2H_{KV} d_h b$$

Predice bytes lógicos bajo supuestos.

MEDICIÓN EMPÍRICA

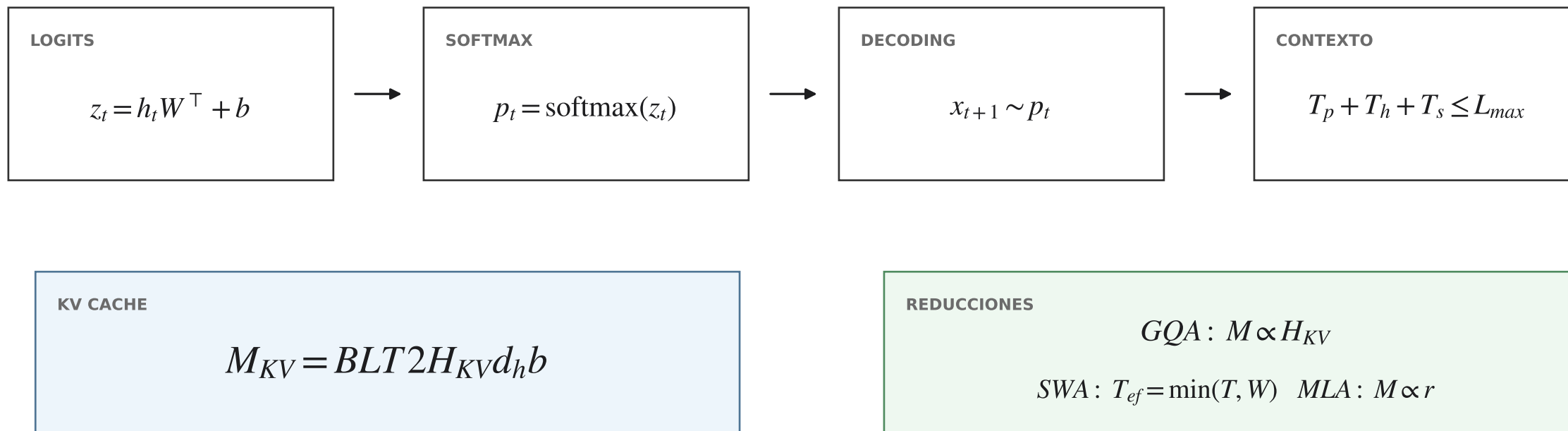
$$\bar{t} = \frac{1}{R} \sum_{r=1}^R t_r$$
$$s = \sqrt{\frac{1}{R-1} \sum_r (t_r - \bar{t})^2}$$

Calentamiento + repetición + sincronización.

Lectura: menos memoria estimada no demuestra automáticamente menor latencia o mayor throughput.

19. Mapa final de ecuaciones - Semana 2

De logits a generación y de generación a eficiencia de inferencia.



Cierre: logits -> distribución -> decoding -> autoregresión -> contexto -> cache -> memoria -> evidencia.

Lectura: Semana 2 conecta la probabilidad del siguiente token con el costo de mantener estado durante inferencia.